

CS II Review

COSC 2230 (CS III) Fall 2026 | Augustana U.

Table of Contents

- [1. Objectives](#)
- [2. Preamble: four illusions of the machine \("abstractions"\)](#)
- [3. What's object-oriented programming \(OOP\) about?](#)
- [4. Reference data types, wrappers, and primitive types](#)
- [5. Code along exercises](#)
- [6. Code along example: a fancy "hello world"](#)
- [7. **PRACTICE** Commented Greeting class](#)
- [8. Java vs C++ vs \(modern\) Python](#)
- [9. **PRACTICE** Extended Greeting](#)
- [10. Interfaces and polymorphism](#)
- [11. Immutability](#)
- [12. The final modifier, aliasing bugs, and defensive copy](#)
- [13. Stopwatch](#)
- [14. Testing the Stopwatch class](#)
- [15. Iterative development](#)
- [16. **PRACTICE** Add a reset method to Stopwatch](#)
- [17. Questions and answers](#)
- [18. Glossary](#)

1. Objectives

Topics:

- Reference data types (classes)
- Interface vs. implementation
- Encapsulation and access modifiers
- Java compilation chain
- Using the command-line (shell)
- Autoboxing and unboxing
- Interfaces and polymorphism (dynamic dispatch)
- Immutability and final
- Static vs. instance fields
- Java packages, libraries, jar files
- OpenJDK version and JIT compilation

Programs:

- Greeting.java ("Hello world" class)
- GreetingTest.java (tests Greeting)
- Greeting2.java and GreetingTest2.java (practice)
- Greeter.java (interface), FormalGreeter.java, CasualGreeter.java
- GreeterTest.java (tests Greeter polymorphically)
- currentTime.java
- Timer.java
- Stopwatch.java
- StopwatchReset.java

- `java.util`
- `java.lang.Math`

2. Preamble: four illusions of the machine ("abstractions")

Abstraction	Illusion
Comp arch (I/O)	"The machine is one"
Operating system	"The machine is all yours"
Application (OOP)	"The app is easy to use"
User (you)	"You're the only one for me"

- For each of these, there is an underlying truth

Abstraction	Illusion	Truth
Computer architecture	"The machine is one"	CPU - RAM - NVM
Operating system	"The machine is all yours"	multi-process management
Application (OOP)	"The app is easy to use"	interface vs. implementation
User (you)	"You're the only one for me"	multi-user management

3. What's object-oriented programming (OOP) about?

- Practically, for you right now:
 - You can define your own data type with `class`
 - You can create instances with `new` and call them with dot-notation
 - You can use library types the same way (e.g. `String`)
 - You can compose your own types into a bigger program
 - You can hide implementation and hand clients an interface (API)
- Technically, as a programming paradigm:
 - A way of organizing a program around objects
 - Bundling data and operations on that data instead of procedures that pass passive data structures around
 - An object owns some state and is responsible for keeping it consistent
 - Object exposes a contract (interface) and hides its representation (encapsulation)
 - You get work done by sending messages & which code runs can depend on the object's type
 - The object's type is decided at run time (polymorphism/dynamic dispatch)
 - Types relate by inheritance/subtyping so one piece of client code works with many kinds of object
- Technically, in Java:
 - `class` is a blueprint; `new` builds an object on the heap, used via dot-notation
 - State lives in instance fields, behavior in instance methods; this is the current object
 - A constructor (class name, no return type) sets up the object's initial state
 - Encapsulation is compiler-enforced: `private`, package-private (default), `public`

- Every class implicitly extends `Object` and may implements an interface (a bodiless contract)
- Which implementation runs is picked at run time from the object's real class (dynamic dispatch)
- Non-primitive types are reference types; the eight primitives aren't objects (wrapper classes + autoboxing bridge them)
- `final` means assign-once; `static` means belongs to the class, not an instance
- Objects are freed automatically (garbage collection); classes group into packages and ship as jar files

We'll look at each of these as part of a simple and an extended example, followed by in-class practice exercises and a home assignment to test your understanding (or your memory).

4. Reference data types, wrappers, and primitive types

- What is a **reference [data] type**?

A reference data type is a data type that consists of **data**, **constructors**, and **methods**. The constructor is used to create **objects** of the data type; the **data** signify the **state** of the object, and the **methods** allow to change the object's **behavior**. They are called **reference** types because a variable of that type doesn't hold the object itself - it holds a **reference** to it.

In terms of **memory**: when you create (or **instantiate**) an object of a reference type, the machine keeps track of the object using a reference, a link, or an address to a region of memory. **An object is a state machine.**

Integers (`int`) are also data types, but they are primitive, non-reference data types. They aren't objects at all.

- What about other languages?
 - Primitive vs. reference type is Java (and C#) specific
 - In Python, Javascript, Ruby everything is a reference
 - C/C++ have aliases (`int&`) that are called references
- What's a **wrapper class**?

Every primitive type has a matching **wrapper** class (`int` -> `Integer`, `double` -> `Double`, and so on), and a wrapper **is** a real reference type - it has a constructor, it lives on the heap, and a variable of that type holds a reference to it, not the value itself.

Java blurs the line on purpose: **autoboxing** automatically converts a primitive into its wrapper wherever a reference type is expected (`Integer i = 5;`), and **unboxing** converts back (`int j = i;`). You'll meet this soon: Chapter 4's generic collections, like `Stack<Integer>`, can only ever hold reference types - `Stack<int>` doesn't compile, `Stack<Integer>` does - and autoboxing is what makes `stack.push(5)` work anyway, with no visible conversion in your code.

`Integer.parseInt(String)` is different: it is an explicit **method call** that parses text into a primitive `int`. Autoboxing is an **implicit** conversion the compiler inserts for you, primitive to wrapper, with no method call written anywhere (`Integer i = 5;`).

5. Code along exercises

- To code along use whatever editor/IDE you normally use.

- If you don't have an editor/IDE, either try Emacs (my mojo), or, for quick access, go to [Onecompiler.com/java](https://onecompiler.com/java) - open, free, no ads.
- There is (usually) at least one code along exercise in every class that you need to upload to qualify as participating in the session.

6. Code along example: a fancy "hello world"

- Code along example: a minimal greeting reference type

```
public class Greeting
{
    String greeting;
    Greeting(String s) { greeting = s; }
    String getGreeting() { return greeting; }

    public static void main(String[] args)
    {
        Greeting greet = new Greeting("hello world");
        System.out.println(greet.getGreeting());
    }
}
```

7. PRACTICE Commented Greeting class

- Can you identify all the parts of this class? Enter comments for every line of the program explaining both syntax and semantics.
- Complete the assignment in class and then upload your .java file to Canvas.

```
// Name of a public class with state (data) and behavior (methods)
public class Greeting
{
    // A String type variable that contains the instance state
    // `greeting` is a reference to a String object
    String greeting;

    // A constructor to create a Greeting object with a
    // String type argument that becomes the greeting reference
    // This removes the implicit no-arg default constructor
    Greeting(String s) { greeting = s; }

    // A Greeting accessor/getter method that returns the
    // state of the Greeting object (a kind of behavior)
    String getGreeting() { return greeting; }

    // A quick self-test embedded in the class, and the program
    // entry point that accepts command-line arguments (String type array)
    // method signature is fixed
    public static void main(String[] args)
    {
        // An instance Greeting object with an initial String value
        // `new` allocates object on the heap and runs constructor
        // `greet` is an object reference, not the object itself
        Greeting greet = new Greeting("hello world");
        // Print the state of the Greeting object:
        // `println` is an instance method on a `PrintStream` object
        // prints the argument then the platform line separator
        System.out.println(greet.getGreeting());
    }
}
```

```
}
}
```

- Why is Greeting not static and not void?

A class declaration is not callable so `static` and `void` have nothing to attach to. Only methods have return types. `static` means "belongs to the enclosing class, not an instance (waiting to be constructed)." But a top-level class has no enclosing class.

- What is `this`?

`this` is a reference to the object the constructor (or method) is currently running on:
`this.greeting` means "the greeting field of this object. You don't need it here but you need it in this example: inside the constructor, `greeting` refers to the parameter and you have to qualify it with `this` to reach the field of the same name.

```
Greeting(String greeting) { this.greeting = greeting; }
```

- Is there any point to the class `Greeting`?

It shows the separation of **interface** vs. **implementation** that can only be achieved by defining your own reference types. It also shows how data/state can be separated from methods/behavior. Lastly, it demonstrates how a class can be tested.

- Is there anything that `Greeting` is missing?

What it does not show is the ability to **encapsulate** data by labelling them as `private` (visible only within the class), or `public`, or marking them `final` (unchangeable). Currently, you could also simply change `greeting` with: `greet.greeting = "hello again";` and print it with `System.out.println(greet.greeting);`, because `greeting` is package-private only.

- How could `Greeting` be changed?

To incorporate **encapsulation**, you have to change `greeting` to be `private` and test access with a main program from outside the class (or the package, since `Greeting` sits in `Greeting.java`).

We first separate the main client, `GreetingTest.java`, from the class definition, and change `greeting` to `private` to see what has changed.

We're working on the command-line to remind ourselves of the use of the Java compiler `javac` and the run-time engine `java`.

- Change your program now:
 1. Change the `greeting` field to `private`.
 2. Save the class declaration to `Greeting.java`
 3. Save the main test client to `GreetingTest.java`
 4. Test the client on the command-line

- Solution:

1. `public class Greeting` becomes `Greeting.java`.
2. `public class GreetingTest` becomes `GreetingTest.java`.
3. `Greeting.java` and `GreetingTest.java` are compiled with `javac`.
4. `GreetingTest` is run with `java` (it contains the `main` method).

```
public class Greeting
{
    private String greeting;
    public Greeting(String s) { greeting = s; }
    public String getGreeting() { return greeting; }
}
```

```
public class GreetingTest
{
    public static void main(String[] args)
    {
        Greeting greet = new Greeting("hello world");
        System.out.println(greet.getGreeting());
        // If `greeting` is `private`, this fails at compile-time:
        //greet.greeting = "hello again";
        //System.out.println(greet.greeting);
    }
}
```

Test it on the command-line:

```
cd ../src # change to directory ./src
rm -rf *.class # remove stale compiled .class files
javac Greeting.java GreetingTest.java 2>&1 # Greeting.class/GreetingTest.class
java GreetingTest # run GreetingTest.class with the JVM
```

Hello, World!

8. Java vs C++ vs (modern) Python

Greeting in C++

- C++ is an object-oriented superset of the C language. It is arguably more popular and used more nowadays than Java. C++ compiles to a standalone native executable without any runtime dependencies: self-contained and shippable.
- The Greeting class in C++ is very similar to Java.
 - The I/O and string libraries have to be loaded
 - C++ has sections for access (public and private)
 - The constructor can be initialized with a list
 - Value instead of reference semantics: the object is created directly on the stack, without new.
 - No garbage collector: Greeting is only destroyed automatically when main returns. If you new it, you'd have to delete it.
- The code:

```
#include <iostream>
#include <string>

class Greeting
{
private:
    std::string greeting;

public:
    Greeting(std::string s) : greeting(s) {}
}
```

```
std::string getGreeting() { return greeting; }

};

int main()
{
    Greeting greet("hello world");
    std::cout << greet.getGreeting() << std::endl;
    return 0;
}
```

hello world

hello world

Greeting in Python

- Python is the most popular high level language. It is written in C, and it is an interpreted rather than compiled language - that means that the user does not need to translate the code to machine code. Under the hood, CPython compiles the source to bytecode, which is then interpreted by a virtual machine (like Java, except that there is no compile-time type checking). There are no separate build artifacts (like `.class` or `.jar`). Much slower than both Java and C++.
- The code looks very different to C++ or Java:
 - No access modifiers like `public` or `private` (that is enforced)
 - Python is dynamically (not statically) typed: object data type is inferred and not explicitly stated.
 - No field declarations up front (like `String greeting`): `__init__` creates `self.greeting` immediately, there is no static type checker enforcing that every instance has this attribute.
 - `self` is explicit everywhere. Java's equivalent (`this`) is implicit unless needed. `self` is required as first parameter of every instance method and used to access fields.
 - `main` is just a function, not special as unique entry point.
 - No `System.out.println`, just `print`. `str()` is called implicitly similar to how Java's `println` calls `toString()`.
- Code:

```
class Greeting:
    def __init__(self, s):
        self.greeting = s

    def get_greeting(self):
        return self.greeting

def main():
    greet = Greeting("hello world")
    print(greet.get_greeting())

if __name__ == "__main__":
    main()
```

hello world

- In modern Python, a `@dataclass` decorator auto-generates system functions like `__init__`. This is close to Java's `record` (a Java 16+ feature).

```

from dataclasses import dataclass

@dataclass
class Greeting:
    greeting: str

    def get_greeting(self):
        return self.greeting

def main():
    greet = Greeting("hello world")
    print(greet.get_greeting())

if __name__ == "__main__":
    main()

```

hello world

9. PRACTICE Extended Greeting

Time = 15 min

1. Write a method setGreeting that sets the greeting value.
2. Write a class GreetingTest2 that tests the new method.
3. Set the greeting to "G'day mate" as command-line input.
4. Upload your
5. When asked, share your screen to show and explain your results.

Solution:

1. Write a method that sets the greeting value.

```

public class Greeting2
{
    private String greeting;
    public Greeting2(String s) { greeting = s; }
    public String getGreeting() { return greeting; }
    public void setGreeting(String s) { greeting = s; }
}

```

2. Write a test in the testing class for the method.

```

public class GreetingTest2
{
    public static void main(String[] args)
    {
        Greeting2 greet = new Greeting2("hello world");
        System.out.println(greet.getGreeting());
        greet.setGreeting("G'day mate");
        System.out.println(greet.getGreeting());
    }
}

```

```
cd ../src
rm -rf *.class
javac Greeting2.java GreetingTest2.java
java GreetingTest2
```

```
hello world
G'day mate
```

3. Set the greeting to "G'day mate" as command-line input.

```
public class GreetingTest3
{
    public static void main(String[] args)
    {
        Greeting2 greet = new Greeting2("hello world");
        System.out.println(greet.getGreeting());
        greet.setGreeting(args[0]);
        System.out.println(greet.getGreeting());
    }
}
```

```
cd src
rm -rf *.class
javac Greeting2.java GreetingTest3.java
java GreetingTest3 "G'day, mate"
```

10. Interfaces and polymorphism

- What does **interface vs. implementation** actually look like in code - not just as a design principle, but as a language mechanism?

A Java interface declares a contract - method signatures with no bodies - that any class can promise to fulfill with `implements`. Once a class implements an interface, you can hold a reference to one of its objects using the **interface** type instead of the concrete class type. Which method body actually runs is decided at **run-time**, based on the real object's class, not the declared type of the variable holding it. This mechanism is called **polymorphism** (or **dynamic dispatch**).

- Why does this matter for what's coming?

Stack and Queue classes are built on exactly this mechanism. Both implement Java's built-in `Iterable<Item>` interface, which requires exactly one method, `iterator()`, returning an object that itself implements `Iterator<Item>` (`hasNext()` and `next()`). That one small interface contract is what lets you write `for (String s : stack)` instead of manually indexing into the stack's internals - the `for-each` loop works by calling `iterator()`, `hasNext()`, and `next()` through the interface, without knowing or caring whether Stack is actually implemented with an array or a linked list underneath. Same idea as Greeter below - just applied to something you'll be building yourself soon.

- A minimal example: one interface, two unrelated implementations.
- First, the contract itself - just a name and a method **signature**, no body at all. This compiles on its own but does nothing yet; it only says "anything that calls itself a Greeter must have a `sayHello()` method that returns a `String`."

```
public interface Greeter
{
    String sayHello();
}
```

- Now the first class that **keeps** that promise. Note implements Greeter in the class header - that's the only thing linking this class to the interface above.

```
public class FormalGreeter implements Greeter
{
    public String sayHello() { return "Good day to you."; }
}
```

- A second class, also implementing Greeter, with a completely different sayHello() body. It has no relationship to FormalGreeter at all - they only share the same contract.

```
public class CasualGreeter implements Greeter
{
    public String sayHello() { return "Hey, what's up?"; }
}
```

- Finally, the client. This is where dynamic dispatch actually happens: the array is declared as Greeter[], not as any concrete class, and the loop calls sayHello() through that interface type - watch for which message prints for which object.

```
public class GreeterTest
{
    public static void main(String[] args)
    {
        Greeter[] greeters = { new FormalGreeter(), new CasualGreeter() };
        for (Greeter g : greeters)
            System.out.println(g.sayHello());
    }
}
```

Compile and run on the command line

```
cd ../src
rm -f *.class
javac Greeter.java FormalGreeter.java CasualGreeter.java GreeterTest.java
java GreeterTest
```

- Neither FormalGreeter nor CasualGreeter knows the other exists - the only thing tying them together is the shared Greeter contract. The loop in GreeterTest never mentions either concrete class by name; g.sayHello() calls whichever implementation the **actual** object in that array slot has, decided at run-time. That's the payoff: code written once against the **interface** keeps working unchanged if you add a third, fourth, tenth Greeter later.
- A note on **abstract classes**: Java has a second mechanism for this same idea, abstract class - a class that, like an interface, can declare methods with no body, but, unlike an interface, can also hold state and provide ready-made methods for subclasses to inherit.

This course's textbook **doesn't use them anywhere** - every polymorphism example in Sedgewick & Wayne is built on **interfaces**. If you see abstract class elsewhere - other courses, other code - it's solving this same problem with a different, heavier tool, not a concept you're missing from here.

11. Immutability

- What does "immutability" mean?

A data type is **immutable** if its values cannot change once created. Since a data type does not have a value (it's a category), we mean the **objects** of that data type: for example, "hello" is the string value of an object of the `String` data type (or class).

- Why does immutability matter?

Immutable objects can be safely used in assignment statements, e.g. `String greeting = "hello";`, and as arguments and return values, e.g. `String greeting(String s)`: even though the callee receives a reference to the very same object, no method exists that lets it mutate that object, so the caller's copy is guaranteed untouched. When we "change" a string argument inside a function, we're not mutating it at all - we're creating a **new** string object; the original string in the calling program is not changed!

Mutable objects don't offer this safety: they can still be passed as arguments and returned, but doing so shares the reference, so a change made through one reference is visible through all of them. E.g. when we pass an array to a function: `sum(int[] a)` and change the array inside the function, the original array in the calling routine is also changed.

- Are primitive types mutable or immutable?

All eight primitive types - `byte`, `short`, `int`, `long`, `float`, `double`, `boolean`, and `char` are immutable, but trivially so: they aren't objects at all, so the question that makes immutability interesting — can another reference alias this and see/cause a change? — doesn't even arise.

For example: `int i = 0;` declares `i` as a variable of primitive type `int`, holding the value `0` directly, not a reference to some object `0` sitting elsewhere. Now `i++`; simply overwrites the value stored in `i` with `1`; nothing that could be aliased by another variable ever changed, because there was never a reference to alias in the first place. Contrast a `String` or array (reference types): there the variable holds a **reference**, and immutability is precisely what decides whether an alias can observe or cause a mutation. That's why the interesting cases in this chapter are all reference types - for a primitive, "immutable" is trivially true but never in question.

12. The final modifier, aliasing bugs, and defensive copy

- To enforce immutability in a data type, you can label it as `final`. A `final` variable is assigned a value only once, either in an inline initialization statement, or in the constructor. Any other code that could modify the value leads to a compile-time error.
- The `final` modifier guarantees immutability only for primitive type instance variables but not for reference types: if an instance variable of a reference type has the `final` modifier, its value (the object reference) will never change - it will always refer to the same object. However the value of the object itself can change.
- If your `final` instance variable is an array, you cannot change the array's length or type, but you can change the values of the array elements. This leads to **aliasing bugs**.
- Example: in the `Vector` class below, `final` only freezes the reference - after the constructor, `cartesian` can never execute again, but if the constructor stores the caller's array by reference instead of copying it, the object and the caller are both aliased to the same array on the heap, and external mutation leaks through to the supposedly immutable type.

```
public class VectorBug
{
    private final double[] cartesian;
```

```

public VectorBug(double[] data) { cartesian = data; } // bug 1: stores
// reference,
// not a copy
public double component(int i) { return cartesian[i]; }
public double[] cartesian() { return cartesian; } // bug 2: leaks
// internal
// reference

public static void main(String[] args)
{
    double[] coords = {4.0, 3.0};
    VectorBug v = new VectorBug(coords);
    System.out.println(v.component(0));
    // caller still holds the same array object
    coords[0] = 999.0;
    // bug 1: 999.0 - v changed with no mutator ever called
    System.out.println(v.component(0));
    // bug 2: mutates v's internal state with no setter
    v.cartesian()[0] = -1.0;
    System.out.println(v.component(0));
}
}

```

- In the code example, the object `v` has no setters, `cartesian` is final, and yet its state changed: two references, one array (bug 1). Mutating through either is indistinguishable to Java. The same hole exists on the way out, if an accessor hands back the live array (bug 2).
- The fix: **defensive copy** at both boundaries - copy on the way in (constructor), and on the way out (any accessor that returns the array) using `data.clone()` (or `Arrays.copyOf`).

```

public class Vector
{
    private final double[] cartesian;
    public Vector(double[] data)
    {
        cartesian = data.clone(); // alternatively:
        // Arrays.copyOf(data, data.length)
    }
    public double component(int i) { return cartesian[i]; }
    public double[] cartesian()
    {
        return cartesian.clone();
    }

    public static void main(String[] args)
    {
        double[] coords = {3.0, 4.0};
        Vector v = new Vector(coords);
        System.out.println(v.component(0));
        // caller still holds the same array object
        coords[0] = 999.0;
        // bug 1 fixed: No access
        System.out.println(v.component(0));
        // bug 2 fixed: No access
        v.cartesian()[0] = -1.0;
        System.out.println(v.component(0));
    }
}

```

- Note that `cartesian` is a private array that only `Vector` itself ever writes to: no outside reference holds a life reference to it in either direction, so `final` actually delivers immutability.
- In the book's `Vector` class, the constructor copies the vector element-wise (rather than with a library call), and the accessor is left out altogether.

- See also: difference between **shallow** and **deep copy** (same thing here, for copy of primitive double content types).
- The textbook code differs in two ways:
 1. the defensive copy is hand-rolled, spelled out loop by loop, rather than using a library function,
 2. The API only exposes `cartesian(int i)`, and never the live `coords` array itself so there is no "bug 2" to defend against.
- This code

```
public class Vector
{
    private final double[] cartesian;
    public Vector(double[] data)
    { // defensive copy on the way in (construction)
        cartesian = new double[data.length];
        for (int i = 0; i < data.length; i++)
            cartesian[i] = data[i];
    }
    public double component(int i) {return cartesian[i];}
    public double[] cartesian()
    { // defensive copy on the way out (access)
        double[] copy = new double[cartesian.length];
        for (int i = 0; i < cartesian.length; i++)
            copy[i] = cartesian[i];
        return copy;
    }

    public static void main(String[] args)
    {
        double[] coords = {4.0, 3.0};
        Vector v = new Vector(coords);
        System.out.println(v.component(0));
        // caller still holds the same array object
        coords[0] = 999.0;
        // bug 1 fixed: no access to `cartesian`
        System.out.println(v.component(0));
        // bug 2 fixed: no direct access
        v.cartesian()[0] = -1.0;
        System.out.println(v.component(0));
    }
}
```

13. Stopwatch

API

- This program is a simplified version of a stopwatch.
- What's an API?

API = Application Programming Interface. For an abstract data type (ADT), like `Stopwatch`, this is a list of all constructors and public methods that can be used to manipulate the state of its objects.

- What do we need to measure time?
 1. Start the stopwatch
 2. Stop the stopwatch
 3. Return the elapsed time

- That settles our minimal API:

```
public class Stopwatch:
```

Returns	Name	Task
	Stopwatch()	Create new stopwatch and start it running
double	elapsedTime()	Return elapsed time since creation (secs)

- What are other things one would like to do with a stopwatch?
 - Reset the stopwatch
 - Pause the stopwatch
 - Start and stop the stopwatch
 - Include a lap timer
 - Swap milliseconds for nanoseconds

Java Utilities and System functions

- Is there already a stopwatch available in Java?

The `java.util` package contains date and time methods, among them `System.currentTimeMillis()`. It returns the number of milliseconds (1/1000 sec) passed since 00:00 hrs on January 1, 1970, the so-called **epoch** or **Unix time**¹.

- Overview of the Java utilities classes in `java.util`:

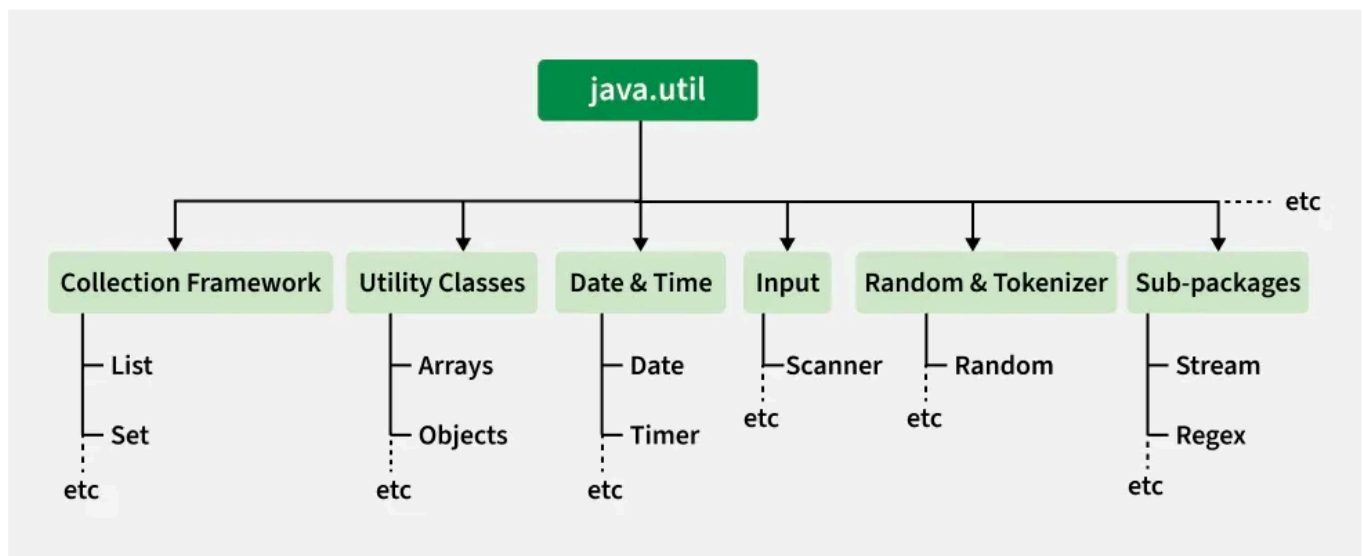


Figure 1: Source: [geeksforgeeks.com](https://www.geeksforgeeks.com/)

- How does it work? A quick program to illustrate (source: [geeksforgeeks.org](https://www.geeksforgeeks.org/))

```
public class currentTime
{
    public static void main(String[] args)
```

```

{
    // Print the milliseconds passed currently
    System.out.println("Milliseconds: " + System.currentTimeMillis());
    // Print the seconds passed currently
    System.out.println("Seconds : " + System.currentTimeMillis()/1000);
    // Print the minutes passed currently
    System.out.println("Minutes : " + System.currentTimeMillis()/1000/60);
    // Print the hours passed currently
    System.out.println("Hours : " + System.currentTimeMillis()/1000/60/60);
    // Print the days passed currently
    System.out.println("Days : " + System.currentTimeMillis()/1000/60/60/24);
    // Print the years passed currently
    System.out.println("Years : " + System.currentTimeMillis()/1000/60/60/24/365);
    // What day is today?
    System.out.println("Today is: " + java.time.LocalDate.now());
}
}

```

- Here is a simple program to time execution:

```

public class Timer
{
    public
    public static void main(String[] args)
    {
        long starting, ending;
        System.out.println("Timing a loop from 0 to 100,000,000");
        starting = System.currentTimeMillis();
        for (long i=0; i < 100000000L; i++);
        ending = System.currentTimeMillis();
        System.out.println("Elapsed time: "+(ending-starting)+" ms.");
    }
}

```

- On the command-line:

```

cd src
javac Timer.java
java Timer

```

The Stopwatch class

- The following test client creates two Stopwatch objects to measure the running time of summing up generalized harmonic numbers with two different methods, `Math.sqrt()` and `Math.pow()`, in order to compare their performance.

```

public class Stopwatch
{
    private final long start;

    public Stopwatch()
    {
        // set stopwatch
        start = System.currentTimeMillis();
    }

    public double elapsedTime()
    {
        // return time between start and now in seconds
        long now = System.currentTimeMillis();
        return (now - start) / 1000.0;
    }
}

```

```

public static void main(String[] args)
{
    int n = Integer.parseInt(args[0]);
    // Compute and time computation using Math.sqrt()
    Stopwatch timer1 = new Stopwatch();
    double sum1 = 0.0;
    for (int i = 1; i <= n; i++)
        sum1 += Math.sqrt(i);
    double time1 = timer1.elapsedTime();
    StdOut.printf("Math.sqrt(): %e (%.2f seconds)\n", sum1, time1);

    // Compute and time computation using Math.pow()
    Stopwatch timer2 = new Stopwatch();
    double sum2 = 0.0;
    for (int i = 1; i <= n; i++)
        sum2 += Math.pow(i, 0.5);
    double time2 = timer2.elapsedTime();
    StdOut.printf("Math.pow(): %e (%.2f seconds)\n", sum2, time2);
}
}

```

14. Testing the Stopwatch class

- Tangle the program and test it for $n = 100,000,000$ and $n = 1,000,000,000$:

```

cd src
rm -rf *.class
javac -cp .:stdlib.jar Stopwatch.java
java -cp .:stdlib.jar Stopwatch 100000000
java -cp .:stdlib.jar Stopwatch 1000000000

```

- To test the effect of the JIT², remove it from the compilation:

```

cd src
rm -rf *.class
javac -cp .:stdlib.jar Stopwatch.java
java -Xint -cp .:stdlib.jar Stopwatch 100000000
java -Xint -cp .:stdlib.jar Stopwatch 1000000000

```

- To test the effect of the improvements JDK 17 to JDK 25:

```

# Select a JDK 25 if one is available, else fall back to the current java.
if [ -s "$HOME/.sdkman/bin/sdkman-init.sh" ]; then          # Linux: SDKMAN
    source "$HOME/.sdkman/bin/sdkman-init.sh"
    sdk use java "$(ls "$HOME/.sdkman/candidates/java/" | grep '^25' | head -1)"
elif /usr/libexec/java_home -v 25 >/dev/null 2>&1; then    # macOS: a JDK 25 is inst
    export JAVA_HOME="$(/usr/libexec/java_home -v 25)"
    export PATH="$JAVA_HOME/bin:$PATH"
else
    echo "No JDK 25 found - running with the current java instead:"
fi
java -version 2>&1

cd src
rm -rf *.class
javac -cp .:stdlib.jar Stopwatch.java

```

```
java -cp .:stdlib.jar Stopwatch 100000000
java -cp .:stdlib.jar Stopwatch 1000000000
```

```
openjdk version "25.0.4.1" 2026-08-18 LTS
OpenJDK Runtime Environment Corretto-25.0.4.8.1 (build 25.0.4.1+8-LTS)
OpenJDK 64-Bit Server VM Corretto-25.0.4.8.1 (build 25.0.4.1+8-LTS, mixed mode, sharing)
```

Under JDK 17, the gap has shrunk to about 2-3% (0.46 vs 0.47 sec). Under JDK 25, re-running the identical program showed **no** measurable difference at all. The likely explanation: newer JITs increasingly recognize common special cases - such as a compile-time-constant exponent of 0.5 - and rewrite `pow(x, 0.5)` to reuse the `sqrt` code path directly. So the honest lesson isn't "`pow()` is always much slower than `sqrt()`" (true in 2017, no longer true here) - it's that performance claims are tied to a specific JVM version, and Stopwatch is precisely the tool that lets you check rather than assume.

15. Iterative development

I offer this without too much extra explanation - will merge it into the class presentation: Below you find an example of iterative development of Stopwatch when I tried to develop it without peeking at the textbook solution (and also without using the book's `stdlib.jar`).

This is the way I develop all my programs - iteratively:

1. Write pseudocode to spec
2. Write code with fixed values, no external input
3. Add command-line input
4. Test client development
5. Summarize lessons learnt

Pseudocode:

```
// Define Stopwatch timer that measures elapsed_time for computations.
// Declare a start_time for the Stopwatch
// Define a Stopwatch constructor that sets start_time
// Define a Stopwatch accessor that returns the elapsed_time
// Define a Stopwatch main test method
// Create a Stopwatch timer at start_time
// Perform a computation
// Compute and print elapsed_time as difference between now
// and start time
```

Code - without external input

```
// Define Stopwatch timer that measures elapsed_time for computations.
public class Stopwatch1
{
    // Declare a start_time for the Stopwatch
    private final double start_time;
    // Define a Stopwatch constructor that sets start_time
    Stopwatch1(double start) { start_time=start; }
    // Define a Stopwatch accessor that returns the elapsed_time
    public double elapsed_time() { return System.currentTimeMillis()-start_time; }
    // Define a Stopwatch main test method
    public static void main(String[] args)
    {
```

```

    // Create a Stopwatch timer at start_time
    Stopwatch1 timer = new Stopwatch1(System.currentTimeMillis());
    // Perform a computation
    for (int i=0; i < 100000000; i++)
        Math.sqrt(i);
    // Compute and print elapsed_time as difference between now
    // and start time
    System.out.println((timer.elapsed_time())/1000.);
}
}

```

Code - with command-line input

```

// Define Stopwatch timer that measures elapsed_time for computations.
public class Stopwatch2
{
    // Declare a start_time for the Stopwatch
    private final double start_time;
    // Define a Stopwatch constructor that sets start_time
    Stopwatch2(double start) { start_time=start; }
    // Define a Stopwatch accessor that returns the elapsed_time
    public double elapsed_time() { return System.currentTimeMillis()-start_time; }
    // Define a Stopwatch main test method
    public static void main(String[] args)
    {
        // Get input: number of iterations n
        long n = Integer.parseInt(args[0]);
        // Create a Stopwatch timer at start_time
        Stopwatch2 timer = new Stopwatch2(System.currentTimeMillis());
        // Perform a computation
        for (int i=0; i < n; i++)
            Math.sqrt(i);
        // Compute and print elapsed_time as difference between now
        // and start time
        System.out.println((timer.elapsed_time())/1000.);
    }
}

```

```

cd src
rm -rf Stopwatch*.class
javac Stopwatch2.java
java Stopwatch2 100000000

```

Code - with command-line input and another timer

```

// Define Stopwatch3 timer that measures elapsed_time for computations.
public class Stopwatch3
{
    // Declare a start_time for the Stopwatch3
    private final double start_time;
    // Define a Stopwatch3 constructor that sets start_time
    Stopwatch3(double start) { start_time=start; }
    // Define a Stopwatch3 accessor that returns the elapsed_time
    public double elapsed_time() { return System.currentTimeMillis()-start_time; }
    // Define a Stopwatch3 main test method
    public static void main(String[] args)
    {
        // Get input: number of iterations n
        long n = Integer.parseInt(args[0]);
        // Create a Stopwatch3 timer at start_time

```

```

Stopwatch3 timer1 = new Stopwatch3(System.currentTimeMillis());
// Perform a computation
for (int i=0; i < n; i++)
    Math.sqrt(i);
// Compute and print elapsed_time as difference between now
// and start time
System.out.println((timer1.elapsed_time())/1000.);
// Create another Stopwatch3 timer at start_time
Stopwatch3 timer2 = new Stopwatch3(System.currentTimeMillis());
// Perform a computation
for (int i=0; i < n; i++)
    Math.pow(i,0.5);
// Compute and print elapsed_time as difference between now
// and start time
System.out.println((timer2.elapsed_time())/1000.);
    }
}

```

```

cd src
rm -rf Stopwatch3.class
javac Stopwatch3.java
java Stopwatch3 100000000

```

Comparison with the textbook code:

- **Constructor design differs.** The book's `Stopwatch()` constructor takes no arguments and calls `System.currentTimeMillis()` internally, guaranteeing every `Stopwatch` object records "now" the moment it's created. `Stopwatch3`'s constructor takes `start` as a parameter, so the **caller** is responsible for passing `System.currentTimeMillis()` correctly at every call site - the class trusts the caller instead of enforcing the guarantee itself.
- **Units are ambiguous at the API level.** `elapsed_time()` returns raw milliseconds (`System.currentTimeMillis() - start_time`, no division); the `/1000.` conversion to seconds happens externally, at each `println` call in `main`. The book's `elapsedTime()` performs the `/1000.0` conversion **inside** the method, so the name and the unit agree, and every caller gets seconds for free, with no risk of forgetting the conversion.
- **The loop discards its result.** Neither the `sqrt` loop nor the `pow` loop stores anything - `Math.sqrt(i)`; and `Math.pow(i,0.5)`; are called purely for their (nonexistent) side effects. The book instead accumulates `sum1 + Math.sqrt(i)` and `sum2 + Math.pow(i,0.5)`, both to have something meaningful to print and to give the JIT no legal excuse to eliminate the loop as dead code.
- **The measured slowdown differs from the book's modern rerun.** Your run: `0.026` sec (`sqrt`) vs `0.075` sec (`pow`) - `pow` about 3x slower - closer to the book's original 2017 finding (over 10x) than to near-parity. Since your loop discards its result, it's theoretically JIT-eliminable, yet the observed times are clearly non-zero and non-trivial - in practice HotSpot did not eliminate either loop here.

The textbook program

```

public class Stopwatch4
{
    private final long start;
    public Stopwatch4() { start = System.currentTimeMillis(); }
    public double elapsed_time()
    {
        long now = System.currentTimeMillis();
        return (now - start)/1000.0;
    }
    public static void main(String[] args)
    {
        int n = Integer.parseInt(args[0]);
    }
}

```

```

Stopwatch4 timer1 = new Stopwatch4();
double sum1 = 0.0;
for (int i=0; i<n; i++)
    sum1 += Math.sqrt(i);
double time1 = timer1.elapsed_time();
System.out.printf("%e (%.2f seconds)\n", sum1, time1);

Stopwatch4 timer2 = new Stopwatch4();
double sum2 = 0.0;
for (int i=0; i<n; i++)
    sum2 += Math.pow(i,0.5);
double time2 = timer2.elapsed_time();
System.out.printf("%e (%.2f seconds)\n", sum2, time2);
    }
}

```

```

cd ../src
rm -rf *.class
java -version 2>&1 | grep "version"
javac Stopwatch4.java
java Stopwatch4 1000000000
java -Xint Stopwatch4 1000000000 # without JIT optimization

```

```

openjdk version "25.0.4.1" 2026-08-18 LTS
2.108185e+13 (0.50 seconds)
2.108185e+13 (0.57 seconds)
2.108185e+13 (6.42 seconds)
2.108185e+13 (6.94 seconds)

```

16. PRACTICE Add a reset method to Stopwatch

Time = 10 min

1. Copy Stopwatch to a new class StopwatchReset, and remove final from start.
2. Write a method reset() that sets start back to the current time, so the same object can be timed a second time without creating a new Stopwatch.
3. Test it: run a task, print elapsedTime(), call reset(), then print elapsedTime() again immediately - it should now read close to zero.
4. Be ready to explain: why did start have to stop being final for this to work?

Solution

```

public class StopwatchReset
{
    private long start; // no longer final - reset() reassigns it

    public StopwatchReset()
    {
        start = System.currentTimeMillis();
    }
    public double elapsedTime()
    {
        return (System.currentTimeMillis() - start) / 1000.0;
    }
    public void reset()
    {
        start = System.currentTimeMillis();
    }
}

```

```

    }
    public static void main(String[] args)
    {
        StopwatchReset timer = new StopwatchReset();
        for (int i = 0; i < 1000000000; i++) Math.pow(i, 0.5);
        System.out.println("before reset: " + timer.elapsedTime());
        timer.reset();
        System.out.println("right after reset: " + timer.elapsedTime());
    }
}

```

Compile and run:

```

cd ../src
rm -rf StopwatchReset.class
javac StopwatchReset.java
java StopwatchReset

```

```

before reset: 0.273
right after reset: 0.0

```

17. Questions and answers

1. What's the meaning of `public static void main(String[] args)?`

1. `public`: this function can be seen and called from anywhere in the program.
2. `static`: this function belongs to the `class` itself, not to any particular object of it, so it can be called without first creating an instance (needed here because no object exists yet)
3. `void`: this function has no return value
4. `main`: this is the mandatory name for the program's entry point
5. `String[] args`: this is an array of strings representing the (optional) arguments to the program on the command-line

2. Why can't a constructor be called `public void Greeting(String s)` if it does not return anything?

A constructor has no return type at all - not even `void`. Java recognizes a constructor by two things together: the name exactly matches the class name, and there is no return type in front of it. Adding `void` turns it into an ordinary method that happens to share the class's name (Java allows that) - it is no longer a constructor.

Since `Greeting` would then have zero constructors, `javac` silently supplies its own implicit no-argument one: `Greeting()`. The line `Greeting greet = new Greeting("hello world");` from the code-along would now fail to **compile** - "constructor `Greeting` in class `Greeting` cannot be applied to given types" - because no constructor accepts a `String` anymore. The `void` method would have to be called separately, like any other method, after first constructing an (uninitialized) object with `new Greeting(): greet.Greeting("hello world");`

3. Why is `start` declared `final`?

`start` is set once, in the constructor, and should never change for the rest of the object's life - the whole point is that it records a fixed moment in time. Declaring it `final` makes the compiler enforce that guarantee, instead of just trusting that no method ever reassigns it by accident.

4. Why can't we just reuse the `timer1` object for `time2`?

Because `start` is `final`. To reuse the timer object, we would need to add a method that can reset the time (home assignment).

5. What if we wanted to run this with fixed `N` (as a constant)?

Add `private static final int N = 100000000;` before `start`. The difference is that `start` is an **instance** field (every `Stopwatch` object gets its own `start`) while `N` is static because it is available globally across the class.

6. At which decimal are the first two results beginning to differ?

Reading the `%.2f` output directly: `0.46` vs `0.47` (and `4.55` vs `4.70`) - both agree at the tenths place and first diverge at the hundredths place, the second decimal digit.

7. Why are the results for `Math.sqrt()` and `Math.pow()` different?

They're barely different, and that's itself the interesting result. The book (2017) reports `Math.sqrt()` as more than 10x faster than `Math.pow(x, 0.5)` - a huge gap, because `Math.sqrt()` is a narrow, specialized operation the JIT can optimize heavily (close to one hardware instruction), while `Math.pow(x, y)` has to stay correct for **any** exponent `y`, including negative, fractional, infinite, and NaN values - that generality costs real time.

8. Why can't I drop the parameters from `main(String[] args)` if I don't run the program with command line arguments?

Because the JVM finds the entry point by reflection, searching for one exact signature: `public static void main(String[] args)` - public, static, void, exactly one `String[]` parameter. That's a fixed runtime lookup, not a flexible match.

`public static void main()` still **compiles** fine - it's a legal method, and `javac` has no opinion about entry points. But it's no longer the entry point: `java ClassName` searches for the exact signature, fails to find it, and the program never runs, whether or not the body ever uses `args`. (C++ can allow a no-argument `main()` because the compiler treats `main` as a small family of sanctioned signatures; Java has no such special-casing - there's exactly one accepted shape.)

9. What's the largest number you can use for `n`?

`args[0]` is parsed with `Integer.parseInt()` into an `int`, and the loop counter `i` is also an `int` - so the hard ceiling is `Integer.MAX_VALUE = 231 - 1 = 2,147,483,647`. Pass anything larger and `Integer.parseInt()` throws a `NumberFormatException` before the program even starts. (Even if `n` were widened to `long`, `i` would still overflow past `Integer.MAX_VALUE`, wrap to a negative number, and the loop condition `i <= n` would behave incorrectly - so the real constraint is the `int` loop counter, not just the parsed argument.)

Below that hard limit, there's also a practical limit: how long you're willing to wait. `n = 1,000,000,000` already took over 4 seconds here; `Integer.MAX_VALUE` would take roughly twice that. (Home assignment)

10. Why wrap the available function into a `Stopwatch` class?

The general answer: Data abstraction pays for itself. Concretely:

- Intent at the call site. `timer1.elapsedTime()` says what you mean. The raw version makes every reader re-derive "oh, this is measuring elapsed time" from a subtraction and a `/1000.0`.
- The arithmetic is written once, not copy-pasted everywhere. Every client that times something would otherwise repeat `(now - start) / 1000.0` — and one client eventually

forgets the `/1000.0` or gets the subtraction order backwards. Centralizing it means one place to get right.

- The representation can change without breaking clients. `currentTimeMillis()` is wall-clock time, which can actually jump if the system clock gets adjusted (NTP sync, etc.) — `System.nanoTime()` is the more correct tool for measuring elapsed intervals since it's monotonic. If you ever want to fix that, change one line inside `Stopwatch`; every client using the class is unaffected.
- You get independent instances for free. The demo runs `timer1` and `timer2` simultaneously, each tracking its own start time. Doing that with raw `currentTimeMillis()` calls means manually juggling `start1/start2` variables yourself with no grouping — the object gives you that bookkeeping automatically.

11. What has changed since JDK 17 (now: JDK 25)?

Both `Math.sqrt()` and `Math.pow()` are HotSpot **intrinsic**s (hand-optimized, often assembly-level implementations the JIT substitutes for the plain Java method). What changed between 17 and 25 is the **quality** of the `pow` intrinsic itself: newer JIT releases regularly refine these hand-written fast paths (e.g. detecting more special-case exponents, using newer CPU instructions). Pinning down the exact JDK release and bug/JEP responsible would mean searching the OpenJDK issue tracker (JBS) directly, not general web search.

We have solid **empirical** evidence (two reproducible runs) that something changed, without a confirmed **mechanistic** citation for exactly what or when.

12. What does the JIT do exactly?

The JIT (Just-In-Time compiler) is the part of the JVM that turns bytecode into native machine code **while the program runs**, instead of all at once ahead of time like `javac` does for source-to-bytecode.

Java source (`.java`) compiles to bytecode (`.class`) once, ahead of time. But the JVM doesn't run that bytecode as machine code directly - it starts out **interpreting** it, instruction by instruction, which is slow. As the JVM runs, it profiles which methods get called a lot (like `Math.pow()` called 100 million times in a loop) and marks them "hot." The JIT then compiles just those hot methods down to optimized native machine code and swaps that in for all future calls - the rest of the program stays interpreted, since compiling code nobody runs repeatedly would waste time for no benefit.

This is also where **intrinsic**s come in (see Q11): for a handful of common methods, HotSpot doesn't even bother compiling the normal Java implementation - it substitutes a hand-written, highly optimized native version directly. `Math.sqrt()` and `Math.pow()` are both on that list, which is why their relative performance can shift between JDK releases as those hand-written versions get refined - the underlying Java source for `Stopwatch` or your test client never changes at all.

13. Can you explain what this code does?

```
source $HOME/.sdkman/bin/sdkman-init.sh
sdk use java 25.0.1-tem
java -version 2>&1
```

Line 1: loads SDKMAN's shell functions (`sdk`, `sdk use`, `sdk install`, ...) into the **current** shell process. `source` runs the script's commands directly in this shell rather than a child process, so whatever it defines stays defined afterward - a plain execution (`./sdkman-init.sh`) would lose everything the moment that child process exited.

Line 2: switches which java/javac this shell's PATH resolves to - here, the installed candidate named 25-open - for the rest of **this shell session only**. It does not touch sdk default, so nothing changes outside this one process.

Line 3: prints the active JVM's version banner, to confirm the switch actually took effect before doing anything else. The 2>&1 matters: java -version writes its output to **stderr**, not stdout (an old convention predating -version being a well-behaved flag) - without redirecting stderr into stdout, some tools/contexts that only capture stdout would show nothing at all, even though the command ran correctly.

14. When does a nested class need to be declared **static**, and when does that question not even apply?

A non-**static** nested class carries a hidden reference to one specific instance of its **outer** (enclosing) class - it can only be created via `outer.new Inner()`, from code that already has an outer instance to attach to. If the code creating it runs in a **static** context (like `main()`), no instance of the outer class exists yet, so there is nothing for that hidden reference to point to. Marking the nested class **static** removes the hidden reference entirely, letting it be created on its own - `new Inner()` - just like an ordinary class.

A **top-level** class (one that isn't nested inside anything at all) never needs this modifier, because the question "does it need a reference to its outer class" doesn't even arise: there is no outer class in the first place. (Note: "outer class" here is unrelated to **superclass** - there's no extends or inheritance involved at all.)

15. Why can a recursive method eventually crash with a `StackOverflowError`, while an equivalent iterative (loop-based) method never does, no matter how large the input?

Each recursive call pushes a new **stack frame** - a region of the call stack holding that call's parameters, local variables, and return address. None of these frames are freed until the base case is reached and the whole chain of calls starts unwinding, returning values back up. The deeper the recursion goes before hitting the base case, the more frames pile up simultaneously, all still "open." The call stack has a fixed size (adjustable with the `-Xss` flag, but always finite) - so with enough recursion depth, it runs out of room: `StackOverflowError`.

A loop-based version is different: the method containing the loop is called **once**, creating exactly **one** stack frame. The loop body runs many times, but that's just repeated execution of statements inside that single, already-existing frame - no new frame is pushed per iteration. Memory usage stays flat regardless of how many times the loop runs, which is why an iterative solution can handle inputs far larger than a recursive one before hitting any hard limit.

16. When javac compiles cleanly (no errors, `.class` files produced), what exactly does that guarantee - and what does it **not** guarantee?

A clean compile means the code passed every **compile-time** check:

- **Syntax** - correct grammar (braces, semicolons, ...).
- **Type checking** - you can't assign a `String` to an `int`, or pass an argument of the wrong type.
- **Name resolution** - every identifier (variable, method, class) must be declared or importable.
- **Compile-time-checkable access rules** - e.g. calling a private member from outside its class.
- A few structural rules - e.g. every path through a non-void method must return something; code placed after a return is unreachable and rejected.

It does **not** guarantee:

- Correct **logic** - the program can compile cleanly and still compute the wrong answer.
- Freedom from **run-time** errors - things like `NullPointerException`, `ArrayIndexOutOfBoundsException`, or `StackOverflowError` are never caught at compile time, only when the running program actually hits them.

"javac success" means **valid, type-safe Java** - not "this program does what I intended."

18. Glossary

Term	Definition
reference type	variable holds a link to the object
primitive type	variable holds the value directly
value semantics	variable holds the object itself, not a link (C++)
reference semantics	variable holds a reference to the object (Java)
statically typed	types checked at compile-time (Java, C++)
dynamically typed	types checked at run-time (Python)
object	an instantiated instance of a class
instance variable	a field holding part of an object's state
constructor	method that creates and initializes objects
record	Java 16+ class auto-generating ctor/accessors
dataclass	Python decorator auto-generating <code>__init__</code> etc.
encapsulation	hides internal state behind public methods
interface	the public contract a type exposes to its callers
implementation	the private internal details behind that contract
API	a type's list of public constructors and methods
private	accessible only inside its own class
package-private	accessible only within the same package (default)
public	accessible from any class
compile-time	checked/enforced by <code>javac</code> , before the program runs
run-time	checked/enforced by the JVM, while the program runs
static	belongs to the class, not an instance
nested class	a class declared inside another class
outer class	the class containing a nested class (not superclass)
void	method returns no value
main	program's mandatory entry-point method
entry point	the method the JVM looks for to start a program

Term	Definition
args	command-line arguments as string array
immutable	state cannot change after construction
mutable	state can change after construction
final	variable can be assigned only once
aliasing	two references pointing at one object
defensive copy	copy made to prevent aliasing bugs
shallow copy	copies references, not the objects
deep copy	copies the objects, not just refs
garbage collector	automatic reclaiming of unused heap memory
package	named grouping of related classes
library	reusable collection of packages/classes
jar file	packaged archive of compiled classes for reuse
bytecode	compiled <code>.class</code> instructions the JVM runs
JIT	compiles bytecode to native code at runtime
intrinsic	hand-optimized native swap-in for a method
epoch (Unix time)	seconds/ms since Jan 1 1970, 00:00 UTC
widening	implicit conversion to a larger/wider type
overflow	value exceeds its type's range and wraps
call stack	fixed-size structure tracking active method calls
stack frame	per-call region holding params/locals/return addr
recursion	a method calling itself on a smaller subproblem
base case	the case that stops a recursion from calling again

Footnotes:

¹ Unix time originated as the system time of Unix operating systems. It differs from universal time (UTC) in its handling of leap seconds so that every day contains exactly 86,400 seconds (POSIX standard), but UTC and Unix time give you the same wall-clock time. Leap seconds are occasional 1-sec adjustments to UTC due to long-term slow-down in the Earth's rotation.

² JIT = Just-In-Time compiler, the part of the JVM that translates Java bytecode (`.class`) into native machine code while the program is running rather than all at once ahead of time. The JIT can compile the operation down to optimized native machine code on the fly - if an operation is "hot" like `Math.pow`, it swaps in the fast version. It can also recognize that a call has always exponent 0.5 and rewrite it to reuse `Math.sqrt()` code instead. The flag `-Xint` switches JIT off.

Author: Marcus Birkenkrahe

Created: 2026-09-04 Fri 11:45